

Pattern Recognition

최종 팀프로젝트 보고서



팀번호	16	과목	Pattern Recognition(02)
제출날짜	2026-05-31		
팀원 (이름/학번)	조수현	2276309	
	김유림	2391046	
	이현민	2476255	
	유다연	2491052	

1. 데이터 탐색

1.1 데이터셋 기본 구조 및 클래스 분포

본 프로젝트에서 사용한 데이터셋은 총 **39,073**개의 샘플과 **15**개의 변수로 구성되어 있으며, 이 중 **14**개는 특징 변수(**Feature**), **1**개는 타겟 변수(**Target**)인 **income**이다.

타겟 변수의 분포를 분석한 결과, 연간 소득이 **50K 이하($\leq 50K$)** 인 샘플은 **29,729개(76.09%)**, **50K 초과($>50K$)** 인 샘플은 **9,344개(23.91%)**로 나타났다. 즉, 약 3:1 비율의 클래스 불균형이 존재함을 확인할 수 있었다.

이러한 결과는 모델 학습 과정에서 다수 클래스에 편향될 가능성이 있음을 의미한다. 따라서 모델 성능 평가 시 단순 정확도(**Accuracy**)뿐 아니라 **Precision, Recall, F1-score**와 같은 불균형 데이터에 적합한 평가지표를 함께 고려할 필요가 있다.

1.2 수치형 변수 탐색

수치형 변수와 소득 수준 간의 관계를 파악하기 위해 **Pearson** 상관계수를 분석하였다. 분석 결과, **education_num(0.335)** 이 가장 높은 상관관계를 보였으며, 그 뒤를 이어 **age(0.230)**, **hours_per_week(0.227)**, **capital_gain(0.223)**, **capital_loss(0.145)** 순으로 나타났다.

특히 교육 수준을 나타내는 **education_num**은 소득과 가장 밀접한 관련성을 보였다. 교육 기간이 **15(전문대학원 수준)**인 집단의 고소득 비율은 **74.53%**, 교육 기간이 **16(박사 수준)**인 집단의 고소득 비율은 **72.94%**로 나타나 고학력일수록 고소득 계층에 속할 가능성이 크게 증가하는 경향을 확인할 수 있었다.

연령(**age**)의 경우 저소득 집단의 평균 연령은 **36.88**세, 고소득 집단의 평균 연령은 **44.27**세로 조사되었다. 이는 사회적 경험과 경력이 축적될수록 소득 수준이 상승하는 경향을 보여준다. 다만 연령 증가에 따른 소득 상승은 일정 시점 이후 완만해지는 비선형적 패턴 또한 관찰되었다.

capital_gain 변수는 대부분의 값이 **0**에 집중되었다. 그러나 최대값인 **99,999**를 기록한 **206**명의 샘플은 모두 고소득 집단으로 분류되었다. 이는 특정 조건이 만족될 경우 소득 클래스가 매우 강하게 결정되는 규칙이 존재함을 보인다.

1.3 범주형 변수 탐색

범주형 변수에 대해서는 **Crosstab** 분석을 수행하여 각 범주별 고소득 비율을 확인하였다.

(1) 혼인 상태(**Marital Status**)

Marital Status	고소득(>50K) 비율
Married-civ-spouse	44.43%

Never-married	4.59%
---------------	-------

기혼자(Married-civ-spouse)는 고소득 비율이 44.43%로 매우 높게 나타난 반면, 미혼자(Never-married)는 4.59%에 불과하였다. 이는 혼인 여부가 소득 수준을 예측하는 데 중요한 변수임을 보여준다.

(2) 직업군(Occupation)

직업군	고소득(>50K) 비율
Exec-managerial	47.59%
Prof-specialty	45.18%
Other-service	4.12%
Priv-house-serv	1.49%

직업군에 따라 소득 분포의 차이가 크게 나타났다.

- 고소득 비율이 높은 직업
 - Exec-managerial(경영·관리직): 47.59%
 - Prof-specialty(전문기술직): 45.18%
- 고소득 비율이 낮은 직업
 - Other-service(기타 서비스직): 4.12%
 - Priv-house-serv(가사 서비스직): 1.49%

직업 유형은 소득 수준을 결정하는 핵심 요인 중 하나로 판단된다.

(3) 성별(Sex)

성별에 따른 소득 분포를 분석한 결과, 남성의 고소득 비율은 30.35%, 여성은 10.93%로 나타났다. 이는 성별 범주에 따라 소득 클래스의 조건부 확률 차이가 존재함을 의미한다.

1.4 EDA 결과 요약

데이터 탐색 결과, 본 데이터셋은 단순한 선형 관계보다는 특정 조건이 충족될 때 소득 클래스가 명확하게 구분되는 비선형적 특성을 가지고 있음을 확인하였다. 특히 **education_num**, **marital_status**, **occupation**, **capital_gain** 변수는 **income** 예측에 중요한 영향을 미치는 핵심 변수로 나타났다.

또한 범주형 변수 내 카테고리별 편차가 매우 크고, 수치형 변수 역시 왜도가 높은 분포를 보이는 경우가 존재하였다. 따라서 단순 선형 모델보다는 변수 간 상호작용과 비선형 경계를 효과적으로 학습할 수 있는 트리 기반 모델(Random Forest, XGBoost 등)이 높은 성능을 보일 것으로 예상된다.

종합적으로 본 데이터는 클래스 불균형, 범주형 변수의 높은 영향력, 그리고 복합적인 비선형 패턴을 포함하고 있어 이러한 특성을 반영한 모델 설계와 평가 전략이 필요함을 확인하였다.

2. 데이터 전처리

원본 13개 입력 변수 → 최종 31개로 단순 feature 폭증을 피하고, CatBoost가 단일 행으로 학습 불가능한 신호만 선별 추가하였다.

2.1 데이터 클리닝

- 결측 표기 표준화: 정규표현식으로 탐지·치환 → workclass 5.66%, occupation 5.69%, native_country 1.73%
- 공백 정규화: 문자열 컬럼에 `str.strip()` 적용 → " Private "/"Private" 동일 카테고리화
- 결측 보존: mode 대체 미채택, workclass 결측 그룹의 >50K 비율 9.1%(정상 24.8%) → 결측 자체가 강한 음(-)의 신호, workclass·occupation 99.6% 동시 결측 → Unknown 카테고리로 명시 보존하여 CatBoost가 자체 학습하였다.
-

2.2 Base Feature Engineering (6개)

신규 변수	정의	의도
cap_net	capital_gain - capital_loss	순 자본 활동
has_capital_stats	자본 활동 유무 binary	sparse 변수 보완(91.8%가 0)
gain_per_hour	capital_gain / (hours+1)	시간 대비 자본 효율
age_bins	[0,25,35,45,60,100] 구간	생애주기 비선형 패턴
sex_marital	성별 × 결혼상태	강력한 상호작용
edu_occ	학력 × 직업	전문직 구분 강화

2.3 Target Encoding (KFold OOF)

고카디널리티 범주형(occupation, native_country, edu_occ 등)을 그 카테고리의 평균 target 값으로 인코딩하여 CatBoost 내부 인코딩과 다른 신호를 추가하였다.

누수 방지: Training set은 5-fold OOF (각 fold 인코딩 = 나머지 4 fold target 평균)로, Test set은 전체 train 통계로 인코딩하였다.

```
def kfold_target_encode(train_series, target, test_series,
                        n_splits=5, smoothing=10, random_state=42):
    kf = KFold(n_splits=n_splits, shuffle=True, random_state=random_state)
    global_mean = target.mean()
    encoded_train = pd.Series(index=train_series.index, dtype=float)
```

```

for train_idx, val_idx in kf.split(train_series):
    ft_x = train_series.iloc[train_idx]; ft_y = target.iloc[train_idx]
    agg = ft_y.groupby(ft_x).agg(['mean', 'count'])
    smooth = (agg['mean']*agg['count'] + global_mean*smoothing) \
              / (agg['count'] + smoothing)
    encoded_train.iloc[val_idx] = train_series.iloc[val_idx]\
        .map(smooth).fillna(global_mean).values
    agg_full = target.groupby(train_series).agg(['mean', 'count'])
    smooth_full = (agg_full['mean']*agg_full['count'] + global_mean*smoothing) \
                  / (agg_full['count'] + smoothing)
    encoded_test = test_series.map(smooth_full).fillna(global_mean)
return encoded_train, encoded_test

```

Smoothing: $(\text{mean} \times \text{count} + \text{global_mean} \times \text{smoothing}) / (\text{count} + \text{smoothing})$ → 적은 sample 카테고리에는 global mean으로 끌어당김 (shrinkage) → 과적합 방지 효과 smoothing = 10.

적용 대상 (5개): occupation, native_country, edu_occ, sex_marital, workclass.

검증 결과:

신규 변수	Target 상관	Train 범위	Test 범위
sex_marital_te	+0.448	[0.030, 0.453]	[0.036, 0.449]
edu_occ_te	+0.422	[0.018, 0.772]	[0.019, 0.767]
occupation_te	+0.349	[0.020, 0.478]	[0.025, 0.475]
workclass_te	+0.182	[0.087, 0.575]	[0.092, 0.561]
native_country_te	+0.092	분포 거의 동일	분포 거의 동일

Train/Test 분포 일치 → 데이터 누수가 없음을 검증하였다.

2.4 Group Aggregation

CatBoost는 단일 행 정보만 학습하므로 '같은 직업군의 평균 나이' 같은 cross-row 통계는 직접 학습이 불가하므로 그룹 통계를 row-level feature로 추가하였다.

```

def add_group_aggregations(train_df, test_df, group_col, num_cols):
    group_stats = train_df.groupby(group_col)[num_cols].mean() # train에서만 fit
    for col in num_cols:
        new_col = f'{group_col}_mean_{col}'
        train_df[new_col] = train_df[group_col].map(group_stats[col])
        test_df[new_col] = test_df[group_col].map(group_stats[col])
        global_mean = train_df[col].mean()
        train_df[new_col] = train_df[new_col].fillna(global_mean)
        test_df[new_col] = test_df[new_col].fillna(global_mean)
    return train_df, test_df

```

```

agg_specs = [
    ('occupation', ['age', 'education_num', 'hours_per_week']),
    ('native_country', ['education_num', 'hours_per_week']),
    ('workclass', ['age', 'hours_per_week']),
]

```

7개 신규 변수 생성 (예: `occupation_mean_age`), 검증: `occupation_mean_age target` 상관 +0.247, `workclass_mean_age` +0.114, 누수 방지: 통계는 `train`에서만 계산하고 `test`에 없는 카테고리는 `train global mean`으로 대체하였다.

2.5 Train-Test Split

```
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y, shuffle=True
)
```

- Stratified sampling (`stratify=y`)하여 76:24 불균형 비율을 보존한다.
- `random_state=42` 로 고정하여 reproducibility 를 확보한다.
- 결과: Train 31,258 / Validation 7,815

2.6 검토 후 배제한 기법

CatBoost 특성을 고려해 다음 기법은 노이즈만 추가하거나 정보 이득이 0이라 배제하였다.

기법	배제 사유
StandardScaler	트리 기반 — 스케일링 불필요
IsolationForest	outlier flag는 노이즈만 추가
Winsorize / log1p	트리 모델은 단조변환에 불변
is_married, is_higher_edu, cap_gain_extreme binary	기존 변수의 단순 함수 → CatBoost가 자체 학습 가능

따라서, 단순 `binary` 추가 실험 결과 `baseline`과 동일한 성능을 보여주고 정보 이득 0임을 알 수 있다. 이를 통해 CatBoost가 자체 학습 불가능한 `cross-fold·cross-row` 통계만 진정한 성능 향상에 기여한다는 점을 알 수 있다.

2.7 데이터 누수 방지 종합

전 단계에서 `fit(train) → transform(test)` 원칙을 일관 적용하였다.

단계	Train 처리	Test 처리
클리닝·Base FE	함수 적용	동일 함수 적용 (stateless)
Target Encoding	5-fold OOF	전체 train 통계로 인코딩
Group Aggregation	train groupby fit	train 통계로 transform
Train-Val Split	<code>stratify=y, random_state=42</code>	별도 (test는 split 불포함)

검증: Target Encoding 후 `train/test` 분포 범위 및 평균 차이 0.003 이내이므로 누수 없음 보여준다.

최종 산출물: 원본 13개 → 6 base FE + 5 target encoded + 7 group aggregated 추가한 총 31개 변수

3. 모델링

이 프로젝트는 개인의 연 소득이 >50K인지 여부를 예측하는 이진 분류 문제이다. 최종 예측 결과는 <=50K를 0, >50K를 1로 변환하여 학습하였으며, 모델은 각 샘플이 높은 소득 분류에 속할 확률(y_prob)과 최종 분류값(y_cls)이 나타난다.

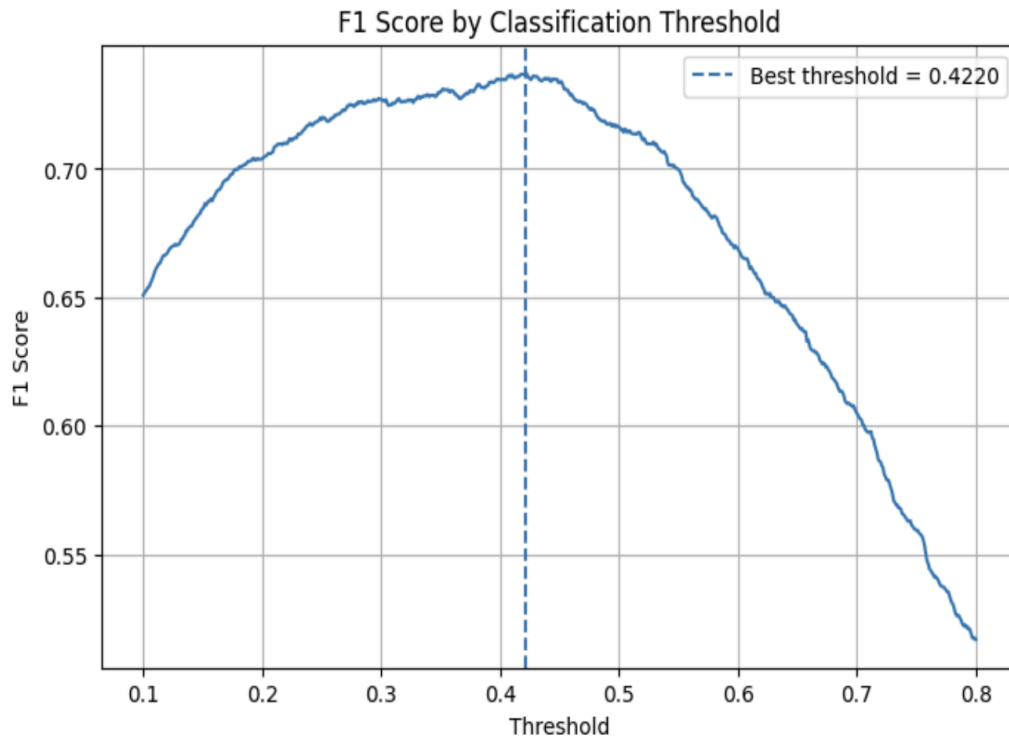
이 프로젝트에서는 모델로 CatBoost를 사용하였다. CatBoost는 Gradient Boosting 계열의 앙상블 모델로, 여러 개의 약한 모델을 순차적으로 학습하면서 강한 모델을 만드는 방식으로 성능을 향상시킨다. 특히 데이터셋은 workclass, education, marital_status, occupation, relationship, race, sex, native_country 와 같은 범주형 변수가 많았기 때문에, 이를 처리하기에 좋은 CatBoost 모델을 선정하였다.

모델 검증을 위해 전체 train 데이터를 stratified holdout 방식으로 train set과 validation set으로 나누었다. stratify=y를 사용하여 소득 클래스의 비율이 train set과 validation set에서 최대한 비슷하게 유지되도록 하였다. 이는 이진 분류 문제에서 클래스 비율이 달라지는 것을 방지하고, 검증 성능을 보다 안정적으로 측정하게 한다.

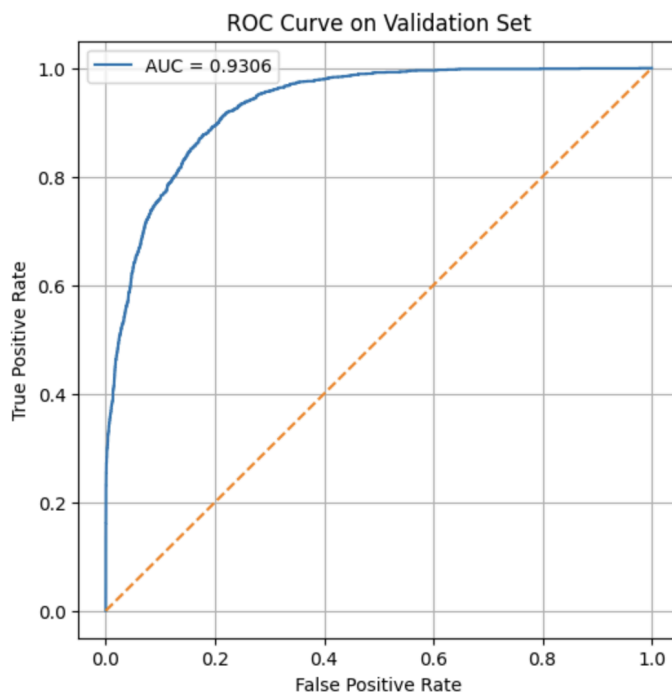
CatBoostClassifier의 주요 하이퍼파라미터는 다음과 같다:
iterations=2500, learning_rate=0.02, depth=7, l2_leaf_reg=10, bagging_temperature=0.8, random_strength=1, border_count=254.

작은 learning rate와 많은 iteration을 사용하여 모델이 점진적으로 학습하도록 하였으며, l2_leaf_reg를 통해 모델 복잡도가 너무 과해지지 않도록 제어하였다. 또한 early_stopping_rounds=200을 적용하여 validation 성능이 일정 기간 개선되지 않으면 학습을 조기 종료하도록 하였다. 이는 train 데이터에 과도하게 적합되는 overfitting을 방지하기 위한 장치이다.

모델의 예측 결과는 확률값으로 출력되므로, 최종 class label을 결정하기 위해 threshold tuning을 수행하였다. 기본 threshold인 0.5를 그대로 사용하지 않고, validation set에서 threshold를 0.1부터 0.8까지 변화시키며 F1 score를 계산하였다. 본 프로젝트의 최종 평가 지표가 (F1 + AUC) / 2이므로, validation AUC와 각 threshold에서의 F1 score를 평균한 값을 기준으로 최적 threshold를 선택하였다.



Validation 결과, 최적 threshold는 0.4220으로 선택되었으며, 검증 AUC는 0.9306, 검증 F1 score는 0.7373, 최종 validation metric $(F1 + AUC) / 2$ 는 0.8339로 나타났다. 최종적으로는 train 전체 데이터를 사용하여 CatBoost 모델을 다시 학습한 뒤, validation set에서 선택한 threshold를 test probability에 적용하여 prediction.csv를 생성하였다.



4. 결론 및 모델 평가

본 프로젝트에서는 고급 피처 엔지니어링(Target Encoding, Group Aggregation)이 적용된 CatBoost 모델의 성능을 극대화하기 위해, 검증 데이터셋을 활용하여 평가 산식에 맞춘 임계값(Threshold) 최적화를 수행하였다.

모델의 종합적인 분류 역량을 보여주는 ROC-AUC와 실전 정확도인 F1-Score의 균형을 맞추고자 두 지표의 평균값을 최종 평가 메트릭으로 설정하였으며, 0.1부터 0.8까지 0.001 간격으로 정밀 탐색을 진행하여 데이터 특성에 가장 적합한 최적의 의사결정 기준을 도출하였다. 임계값 확정 이후에는 전체 학습 데이터로 모델을 다시 학습시켜 테스트 데이터에 대한 예측 확률과 클래스 결과를 성공적으로 생성하였고, 최종 클래스 분포 검토를 통해 특정 클래스에 편향되지 않은 안정적인 예측력을 검증하였다.

결과적으로 기본 기준값(0.5)대신 데이터의 실제 특성을 적절히 반영하고 불균형을 극복함으로써 예측 모델의 전반적인 완성도와 분류 성능을 끌어올렸다.